PROGRAM

package.json

```
{
  "name": "blog-backend",
  "version": "1.0.0",
  "description": "Secure Blog Platform Backend with Express.js, Routing, and JWT Authentication",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "bcrypt": "^5.1.1",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.5.1",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

index.js

```
const express = require('express');
const dotenv = require('dotenv');
const mongoose = require('mongoose');
const cors = require('cors');
const multer = require('multer');
const path = require('path');

dotenv.config();

const authRouter = require('./controllers/auth');
const blogRouter = require('./controllers/blog');
const userRouter = require('./controllers/user');

const app = express();

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URL);
    console.log('DB is successfully connected');
  } catch (error) {
    console.error('Failed to connect to MongoDB:', error.message);
    process.exit(1);
  }
}
connectDB();

app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => {
  res.status(200).json({ msg: 'Blog backend API is running' });
});

app.use('/auth', authRouter);
app.use('/blog', blogRouter);
app.use('/user', userRouter);
app.use('/images', express.static('public/images'));

const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'public/images');
  },
  filename: function (req, file, cb) {
    const uniqueName = Date.now() + '-' + Math.round(Math.random() * 1e9) +
path.extname(file.originalname);
    cb(null, uniqueName);
  }
});
const upload = multer({ storage: storage });
```

```
app.post('/upload', upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ msg: 'No file uploaded' });
  }
  return res.status(200).json({ msg: 'Successfully uploaded', filename: req.file.filename });
});

app.use((req, res) => {
  res.status(404).json({ msg: 'Route not found' });
});

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ msg: err.message || 'Something went wrong' });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server is connected successfully on port ${PORT}`));
```

models/User.js

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema(
  {
    username: { type: String, required: true, unique: true, minlength: 3, maxlength: 20 },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true, minlength: 6 },
    location: { type: String },
    countryOfOrigin: { type: String }
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', UserSchema);
```

models/Blog.js

```
const mongoose = require('mongoose');

const BlogSchema = new mongoose.Schema(
  {
    userId: { type: String, required: true },
    title: { type: String, required: true, minlength: 4 },
    desc: { type: String, required: true, minlength: 12 },
    photo: { type: String, required: true },
    categories: { type: [String], default: [] }
  },
  { timestamps: true }
);

module.exports = mongoose.model('Blog', BlogSchema);
```

middlewares/verifyToken.js

```
const jwt = require('jsonwebtoken');

function verifyToken(req, res, next) {
  const authHeader = req.headers.token;

  if (!authHeader) {
    return res.status(401).json({ msg: 'You are not authenticated' });
  }

  const token = authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ msg: 'Malformed authorization header' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ msg: 'Token is not valid' });
    }
    req.user = user;
    next();
  });
}

module.exports = verifyToken;
```

controllers/auth.js

```
const authRouter = require('express').Router();
const User = require('../models/User');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

authRouter.post('/register', async (req, res) => {
  try {
    const { username, email, password } = req.body;

    if (!username || !email || !password) {
      return res.status(400).json({ msg: 'username, email and password are required' });
    }

    const isExisting = await User.findOne({ email });
    if (isExisting) {
      return res.status(409).json({ msg: 'Already an account with this email. Try a new email' });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const newUser = new User({ username, email, password: hashedPassword });
    const savedUser = await newUser.save();

    const { password: _pw, ...others } = savedUser._doc;
    const token = jwt.sign({ id: savedUser._id }, process.env.JWT_SECRET, { expiresIn: '3d' });

    return res.status(201).json({ user: others, token });
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

authRouter.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ msg: 'email and password are required' });
    }

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({ msg: 'Invalid credentials' });
    }

    const isCorrect = await bcrypt.compare(password, user.password);
    if (!isCorrect) {
      return res.status(400).json({ msg: 'Invalid credentials' });
    }

    const { password: _pw, ...others } = user._doc;
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '3d' });

    return res.status(200).json({ user: others, token });
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

module.exports = authRouter;
```

controllers/blog.js

```
const blogRouter = require('express').Router();
const verifyToken = require('../middlewares/verifyToken');
const Blog = require('../models/Blog');

blogRouter.post('/', verifyToken, async (req, res) => {
  try {
    const newBlog = new Blog({ userId: req.user.id, ...req.body });
    const savedBlog = await newBlog.save();
    return res.status(201).json(savedBlog);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

blogRouter.get('/getAll', async (req, res) => {
  try {
```

```
    const blogs = await Blog.find({}).sort({ createdAt: -1 });
    return res.status(200).json(blogs);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

blogRouter.get('/:id', async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id);
    if (!blog) {
      return res.status(404).json({ msg: 'Blog not found' });
    }
    return res.status(200).json(blog);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

blogRouter.get('/user/:userId', async (req, res) => {
  try {
    const blogs = await Blog.find({ userId: req.params.userId }).sort({ createdAt: -1 });
    return res.status(200).json(blogs);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

blogRouter.put('/updateBlog/:id', verifyToken, async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id);
    if (!blog) {
      return res.status(404).json({ msg: 'Blog not found' });
    }
    if (blog.userId !== req.user.id) {
      return res.status(403).json({ msg: 'You can update only your own posts' });
    }
    const updatedBlog = await Blog.findByIdAndUpdate(
      req.params.id,
      { $set: req.body },
      { new: true }
    );
    return res.status(200).json(updatedBlog);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

blogRouter.delete('/deleteBlog/:id', verifyToken, async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id);
    if (!blog) {
      return res.status(404).json({ msg: 'Blog not found' });
    }
    if (blog.userId !== req.user.id) {
      return res.status(403).json({ msg: 'You can delete only your own posts' });
    }
    await Blog.findByIdAndDelete(req.params.id);
    return res.status(200).json({ msg: 'Blog deleted successfully' });
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

module.exports = blogRouter;
```

controllers/user.js

```
const verifyToken = require('../middlewares/verifyToken');
const User = require('../models/User');
const userRouter = require('express').Router();

userRouter.get('/find/:userId', async (req, res) => {
  try {
    const user = await User.findById(req.params.userId);
    if (!user) {
      return res.status(404).json({ msg: 'No such user' });
    }
    const { password, ...others } = user._doc;
    return res.status(200).json(others);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
```

```
});

userRouter.get('/findAll', async (req, res) => {
  try {
    const users = await User.find({});
    const formattedUsers = users.map((user) => ({
      username: user.username,
      email: user.email,
      _id: user._id,
      createdAt: user.createdAt,
      updatedAt: user.updatedAt
    }));
    return res.status(200).json(formattedUsers);
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

userRouter.delete('/deleteUser/:userId', verifyToken, async (req, res) => {
  if (req.params.userId !== req.user.id) {
    return res.status(403).json({ msg: 'You can delete only your own profile' });
  }
  try {
    const deleted = await User.findByIdAndDelete(req.params.userId);
    if (!deleted) {
      return res.status(404).json({ msg: 'No such user' });
    }
    return res.status(200).json({ msg: 'Successfully deleted' });
  } catch (error) {
    return res.status(500).json({ msg: error.message });
  }
});

module.exports = userRouter;
```

`.env` *(create in project root, not committed to Git)*

```
MONGO_URL=mongodb://localhost:27017/blogdb
JWT_SECRET=mySuperSecretKey123
PORT=5000
```

RUNNING THE PROJECT

```
npm install
npm run dev
```

Test routes in Postman:

```
POST   http://localhost:5000/auth/register       -> { "username", "email", "password" }
POST   http://localhost:5000/auth/login          -> { "email", "password" }
POST   http://localhost:5000/blog                -> { "title", "desc", "photo", "categories" }
(Header: token: Bearer <JWT>)
GET    http://localhost:5000/blog/getAll
GET    http://localhost:5000/blog/:id
PUT    http://localhost:5000/blog/updateBlog/:id       (Header: token: Bearer <JWT>)
DELETE http://localhost:5000/blog/deleteBlog/:id       (Header: token: Bearer <JWT>)
GET    http://localhost:5000/user/findAll
DELETE http://localhost:5000/user/deleteUser/:userId   (Header: token: Bearer <JWT>)
```

SAMPLE OUTPUT

Terminal, after "npm run dev":

```
DB is successfully connected
Server is connected successfully on port 5000
```

Postman — POST /auth/register body: { "username": "arjun_dev", "email": "arjun@example.com", "password": "Password123" }

```
Status: 201 Created
{
  "user": {
    "_id": "66d1f2a8c9e77a1234567abc",
    "username": "arjun_dev",
    "email": "arjun@example.com",
    "createdAt": "2026-08-29T10:15:32.101Z",
    "updatedAt": "2026-08-29T10:15:32.101Z"
```

```
  },
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY2ZDFmMmE4YzllNzdhMTIzNDU2N2FiYyIsImlhdCI6MTcyNTAwMDAwM
CwiZXhwIjoxNzI1MjU5MjAwfQ.Xy3f9k2Lm8QpR7vN4tA1eB6cD0hJ5sW9uI2oP8qT3yZ"
}
```

Postman — POST /auth/login body: { "email": "arjun@example.com", "password": "Password123" }

```
Status: 200 OK
{
  "user": {
    "_id": "66d1f2a8c9e77a1234567abc",
    "username": "arjun_dev",
    "email": "arjun@example.com",
    "createdAt": "2026-08-29T10:15:32.101Z",
    "updatedAt": "2026-08-29T10:15:32.101Z"
  },
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY2ZDFmMmE4YzllNzdhMTIzNDU2N2FiYyIsImlhdCI6MTcyNTAwMDAwM
CwiZXhwIjoxNzI1MjU5MjAwfQ.Xy3f9k2Lm8QpR7vN4tA1eB6cD0hJ5sW9uI2oP8qT3yZ"
}
```

Postman — POST /blog header: token: Bearer <token from login> body: { "title": "Getting Started with Express.js", "desc": "A beginner friendly walkthrough of setting up an Express server.", "photo": "express-basics.jpg", "categories": ["nodejs", "express"] }

```
Status: 201 Created
{
  "_id": "66d1f3b1c9e77a1234567def",
  "userId": "66d1f2a8c9e77a1234567abc",
  "title": "Getting Started with Express.js",
  "desc": "A beginner friendly walkthrough of setting up an Express server.",
  "photo": "express-basics.jpg",
  "categories": ["nodejs", "express"],
  "createdAt": "2026-08-29T10:20:11.552Z",
  "updatedAt": "2026-08-29T10:20:11.552Z"
}
```

Postman — GET /blog/getAll

```
Status: 200 OK
[
  {
    "_id": "66d1f3b1c9e77a1234567def",
    "userId": "66d1f2a8c9e77a1234567abc",
    "title": "Getting Started with Express.js",
    "desc": "A beginner friendly walkthrough of setting up an Express server.",
    "photo": "express-basics.jpg",
    "categories": ["nodejs", "express"],
    "createdAt": "2026-08-29T10:20:11.552Z",
    "updatedAt": "2026-08-29T10:20:11.552Z"
  }
]
```

Postman — DELETE /blog/deleteBlog/:id (wrong owner or missing token)

```
Status: 403 Forbidden
{ "msg": "You can delete only your own posts" }
```

MongoDB Compass: the "blogdb" database shows two collections, "users" and "blogs", with the documents created above visible under each.